

AEO + GEO → llms.txt Generator — Project Overview

1. What this project is

Given any public website URL, the system:

1. Scrapes the website (including JavaScript-rendered sites)
2. Extracts meaningful content (headings, FAQ, structured data, links, etc.)
3. Scores it for **AEO** (Answer Engine Optimization) and **GEO** (Generative Engine Optimization)
4. Generates a fresh `llms.txt` file from that content
5. Exposes all of it through a single embeddable `<script>` widget

```
<script src="http://localhost:3000/widget.js" data-url="https://example.com"></script>
```

Drop that one tag into any HTML page, and it auto-runs, calls the backend, and renders the results — no build step, no manual wiring.

2. Why this problem matters (the “why” behind AEO/GEO)

Search is splitting into two new channels beyond classic SEO:

- **AEO — Answer Engine Optimization.** Tools like Google Featured Snippets, Siri, Alexa, and voice assistants don’t want to read your whole page — they want to **lift one direct answer out of it**. AEO rewards content structured so a machine can extract a fact cleanly: question-style headings, FAQ blocks, answer-first paragraphs, lists/tables, and structured data (JSON-LD).
- **GEO — Generative Engine Optimization.** LLM-based tools (ChatGPT, Claude, Gemini) don’t quote one page — they **synthesize an answer from many sources** and decide which ones to trust and cite. GEO rewards signals of credibility: Organization/Person/Article schema, author attribution, E-E-A-T signals (Experience, Expertise, Authoritativeness, Trust), internal linking depth, and content freshness.

In one line: AEO = “can a machine extract a fact from this page?” GEO = “does a machine trust this page enough to cite it?”

`llms.txt` itself is an emerging convention (like `robots.txt`, but for AI) — a plain-text file that tells AI crawlers/agents what a site is about, its key pages, and how it’d like to be used/cited. This project generates that file automatically from a real scrape, instead of someone hand-writing it.

3. Why this tech stack

Choice	Why
Node.js + Express	Simple, fast to stand up a single-purpose API; matches “no build step” goal for the widget side too.
Axios + Cheerio	Cheerio parses HTML like jQuery — ideal for pulling structured data (headings, JSON-LD, tables) out of a DOM without a full browser, which is fast and cheap for the common case (traditional server-rendered sites).
Playwright (added later)	Many real sites (React/Vue/Next.js SPAs) send near-empty HTML and build the page with JavaScript <i>after</i> load. Axios+Cheerio alone can’t see that. Playwright drives a real headless Chromium that executes the JS and hands back the fully rendered page — used only as a fallback, so we don’t pay the cost on every request.
Vanilla JS widget (no framework, no bundler)	The deliverable is a single <code><script src="...widget.js"></code> tag any site can drop in. A framework would require a build step and a larger runtime payload for something that’s meant to be a lightweight embed. ES modules + dynamic <code>import()</code> give us clean internal organization without needing Webpack/Vite.
Modular folder structure (<code>engines/</code> , <code>routes/</code> , <code>utils/</code> , <code>services/</code> , <code>models/</code>)	Explicit requirement — keeps scraping, scoring, and generation logic independently testable/extensible instead of one giant file.

4. Architecture at a glance

```
backend/
├── server.js           # Express entry, CORS, serves frontend/ statically
├── routes/            # GET /analyze
├── controllers/       # parses request, calls the service
├── services/
│   ├── analyze.service.js # orchestrator: scrape → score → generate → assemble
│   ├── recommendations.service.js
│   └── cache.service.js   # avoids re-scraping the same URL repeatedly
├── scraper/
│   ├── scraper.js        # fetch → (fallback to headless render if empty) → extract
│   ├── renderPage.js     # Playwright headless-render fallback
│   └── extractors/       # one file per data point (title, FAQ, JSON-LD, author, ...)
├── engines/
│   ├── aeo/              # 9 independent checks + scorer
│   ├── geo/              # 10 independent checks + scorer
│   └── llmsTxt/           # composes the generated llms.txt from 6 sections
├── models/             # shape of Website / CheckResult / AnalysisResult
└── utils/               # http client, url/text helpers, JSON-LD parsing, logger

frontend/
├── widget.js           # the single script tag entry point
├── widget/             # ES modules: apiClient, uiRenderer, components/*
├── styles/widget.css
└── demo/
    ├── index.html      # embed example (hardcoded data-url)
    └── analyzer.html    # paste-a-URL UI for manual testing
```

5. The full request flow

1. Page loads with `<script src="widget.js" data-url="...">`
 - |
2. widget.js auto-runs, reads data-url, calls:
 - GET /analyze?url=<target>
 - |
3. Backend validates the URL, hands off to analyze.service.js
 - |
4. SCRAPE
 - a. Fast path: axios fetches raw HTML → cheerio parses it
 - b. Heuristic check: is the page body basically empty (JS shell)?
 - if yes, re-fetch via Playwright (headless Chromium), which executes the JS and waits for the real content to render
 - c. Extractors pull out: title, meta, headings, nav, FAQ, articles, tables, lists, links, images, JSON-LD, author, dates, org, breadcrumbs → assembled into one "Website Model"
 - |
5. SCORE (two independent engines run on the same Website Model)
 - a. AEO Engine → 9 weighted checks → AEO score (0-100)
 - b. GEO Engine → 10 weighted checks → GEO score (0-100)
 - |
6. GENERATE
 - llms.txt Generator composes 6 sections (summary, topics, products/services, important URLs, categories, AI usage notes) from the same Website Model → one formatted llms.txt string
 - |
7. RECOMMEND
 - Every failed check from both engines → sorted by weight → turned into a prioritized recommendation list
 - |
8. Backend returns one JSON object:
 - { website, scores, checks, recommendations, generatedLlmsTxt }
 - |
9. widget.js renders: score cards, pass/fail checklists, recommendations, and the llms.txt text box with Copy/Download

6. How the score is actually calculated

Each check is a small pure function: `(websiteModel) → { passed, weight, message }`.

```
totalWeight = sum of every check's weight
earnedWeight = sum of weight for checks that passed
score       = round((earnedWeight / totalWeight) * 100)
```

Example — AEO checks and their weights:

Check	Weight
FAQ content	12
Structured data present	12
Question-style headings	10
Answer-first content	10
HowTo schema	8
Q&A content	8
Heading hierarchy	8
Lists	6
Tables	6
Total	80

If a page passes FAQ + structured data + answer-first (34/80) → score = **43**.

GEO uses the same formula with its own 10 checks. **Overall score = (AEO + GEO) / 2**.

Weights aren't arbitrary — they roughly track how strongly each signal is known to influence answer-engine extraction (AEO) or generative-engine trust (GEO); e.g. structured data and FAQ content are weighted highest because they're the most directly machine-readable signals.

7. What's been built and verified so far

-  Full modular backend (scraper, both engines, llms.txt generator, API)
-  Headless-browser fallback for JS-rendered (SPA) sites — verified against a live React site (`thiranx.in`), confirmed the FAQ check went from failing (empty shell) to passing (real content) after the fix
-  Embeddable `widget.js` — verified via automated browser testing (Playwright): loads, no console errors, renders scores/checks/recommendations/llms.txt, Copy and Download buttons both work
-  `demo/index.html` (static embed example) and `demo/analyzer.html` (paste-a-URL manual testing UI)
-  Tested end-to-end against multiple real sites (`example.com` , `stripe.com/docs` , `thiranx.in`) with visibly different, correctly differentiated scores

8. Possible next steps (not yet done)

- Improve FAQ/accordion detection for non-heading-based markup (e.g. `<div>`-based accordions)
- Rate limiting on the `/analyze` endpoint
- Persist results (currently in-memory cache only, cleared on server restart)
- Multi-page crawling (currently analyzes exactly one URL per request, by design — matches the “give it a URL” spec)